

3.3 Developing on Raspberry Pi

When developing software on *embedded devices* such as Raspberry Pi, the development environment doesn't usually run on the device itself, but on the developer's workstation computer. This results in slightly different debugging requirements. To connect the debugger to the device, a network connection must be established with the corresponding port forwarding. If Raspberry Pi is running in a production system without internet access, good logging of the application is important to track down errors later.

We'll address these two topics in the following two sections.

3.3.1 Step-by-Step Debugging on a Raspberry Pi

Classic debugging involves setting *breakpoints* in the code and executing the program with a debugger. As soon as the point with the breakpoint is reached, the execution gets stopped, and it's possible to check the status of the application at this point. In addition, you can then follow each call *step by step* and examine the content of variables.

We haven't found a good application for AI for this activity. It remains manual work to set the breakpoints and check the application status. However, AI can support us with the setup so we can start step-by-step debugging in the first place. We've developed a Python application on a Raspberry Pi that controls a stepper motor and is controlled via an integrated touch display.

Prompt: What tools do I need to debug my Python app step-by-step on a remote Raspberry Pi?

To step-by-step debug your Python app on a remote Raspberry Pi, you'll need the following tools:

1. SSH client: To connect to your Raspberry Pi remotely.
2. Remote debugging tool. There are a few options:
 1. pdb (Python Debugger): Built-in Python debugger.
 2. rpdb: Remote Python Debugger, an extension of pdb.
 3. Visual Studio Code with Remote SSH extension: Provides a full IDE experience.
3. Code editor: If not using VS Code, you'll need a way to view and edit your code.
4. Port forwarding: To allow your local machine to connect to the debugger on the Pi.

Here's a basic setup using rpdb: [more details]

At the end of the instruction, which is not printed here, Claude 3.5 Sonnet mentions that the debugging process is smoother if you use VS Code and the Remote SSH plug-in. We want to know more about this:

Prompt: How would I do it using VS Code with the Remote SSH extension?

Using Visual Studio Code with the Remote SSH extension is indeed a more user-friendly approach for remote debugging. Here's how to set it up:

1. Install Visual Studio Code on your local machine.
2. Install the "Remote - SSH" extension in VS Code.

[more details]

This is followed by 12 steps that briefly explain how to enable debugging for Python on the Raspberry Pi. Even if this sounds time consuming, you'll quickly arrive at a functioning debugging solution.

3.3.2 Help for Creating Log Entries

When debugging devices aren't on-site and are possibly offline, it's good practice to write important messages to a log file. This means that errors can also be discovered at a later date. We worked on a project in which the wireless network can be configured from a cloud platform on Raspberry Pi computers that are connected via the mobile network. A Python applica-

tion on a Raspberry Pi makes the corresponding system calls to permanently save the settings.

We tried to get some support in creating the logging information from GitHub Copilot:

Prompt: Help me debug this function with some logging information.

```
def set_wifi_wpa_cli(ssid, psk):
    if ssid == '' or psk == '':
        return
    basecmd = ['sudo', 'wpa_cli', '-iwlan0']
    cmd = basecmd + ['list_networks']
    subprocess.run(cmd, capture_output=True)
    out_str = res.stdout.decode('utf-8')
    if re.search(rf"\t{ssid}\t", out_str, re.MULTILINE):
        # network already there
    else:
        cmd = basecmd + ['add_network']
        res = subprocess.run(cmd, capture_output=True)
        netnr = res.stdout.decode('utf-8')
        cmd = basecmd + ['set_network', netnr, 'ssid',
                        f'"{ssid}"']
        res = subprocess.run(cmd, capture_output=True)
        cmd = basecmd + ['set_network', netnr, 'psk', f'"{psk}"']
        res = subprocess.run(cmd, capture_output=True)
        ...
```

To add logging information to the `set_wifi_wpa_cli` function, you can use Python's built-in logging module. This will help you trace the execution flow and capture the output of each command.

```
logging.basicConfig(level=logging.DEBUG,
                    format='%(asctime)s - %(levelname)s - %(message)s')
...
if ssid == '' or psk == '':
    logging.warning("SSID or PSK is empty.")
    return
...
```

```
if re.search(rf"\t{ssid}\t", out_str, re.MULTILINE):
    logging.info(f"Network {ssid} already exists.")
else:
    cmd = basecmd + ['add_network']
    logging.debug(f"Running command: {' '.join(cmd)}")
    res = subprocess.run(cmd, capture_output=True)
    netnr = res.stdout.decode('utf-8').strip()
    logging.debug(f"Network number: {netnr}")
...
```

As you can see in the excerpts from the response (we've abridged the output here), Copilot is doing a great job. First, logging is configured for Python. In a further step, the log outputs are analyzed according to their importance and marked accordingly with `info`, `warning`, or `debug`. This makes it easier for us to filter the log file later. For the system calls (`subprocess.run`), both the command and the result are logged. The AI also uses the relatively modern Python notation for character strings with *F-strings*.